

COMP0197 — Group 12 Submission Instructions

Project: Probabilistic UK Electricity Demand Forecasting with LSTM Models **Submission:** single zip file. All data, model weights, and figures required to reproduce the reported results are included inside this folder.

1. Environment

The submission targets the course `comp0197-pt` micromamba environment. The environment ships with Python 3.12, PyTorch 2.10 (CPU build), torchvision and NumPy.

1.1 Activate the environment

```
micromamba activate comp0197-pt
```

1.2 Additional packages (3 of 3 used)

Package	Version tested	Why it is required
<code>pandas</code>	3.0.2	Tabular handling of half-hourly demand and temperature, used throughout <code>src/data_loader.py</code> .
<code>matplotlib</code>	3.10.8	All training-curve, prediction, calibration, and ablation figures in <code>src/evaluate.py</code> and the <code>experiments/</code> scripts.
<code>scipy</code>	1.17.1	<code>scipy.stats.norm</code> for the closed-form Gaussian CRPS in <code>src/evaluate.compute_crps</code> , and <code>scipy.optimize.minimize_scalar</code> for the temperature-scaling search in <code>src/evaluate.find_temperature</code> .

Install command (run after activating the environment):

```
pip install pandas matplotlib scipy
```

No other packages are required. Earlier drafts used `proprscoring`, `holidays`, and `requests`; we replaced them with a closed-form CRPS, a hardcoded UK bank-holiday set in `src/data_loader.py`, and `stdlib urllib.request`, so the additional-package count stays at exactly three.

1.3 Hardware

The submission is designed to run on CPU. PyTorch in `comp0197-pt` is a CPU build, and every training/evaluation script auto-selects `torch.device('cuda' if torch.cuda.is_available() else 'cpu')`. No GPU is required to reproduce any reported number.

1.4 Internet access

Not required at run time. The submission ships:

- `data/processed/data/demanddata_2001.csv ... demanddata_2025.csv` — UK National Grid ESO half-hourly demand (full 25-year history)
- `data/raw/temperature_london.csv` — Open-Meteo London 2 m temperature, hourly, 2001–2025
- `saved_models/*.pt` — final trained weights for all three models
- `saved_models/*.json` — per-model training histories, scalers and calibration scalars

`train.py` retains a `stdlib urllib` based fallback that re-fetches the temperature data from `archive-api.open-meteo.com` if the cache file is ever deleted, but a marker re-running the pipeline never has to hit the network.

2. Quick reproduction — evaluate the shipped weights

This is the fastest way to verify the headline numbers in the report. It loads the three trained models from `saved_models/` and re-runs inference on the held-out test set.

```
micromamba activate comp0197-pt
cd <submission folder>
python test.py
```

Expected runtime: roughly **10–20 minutes on a typical laptop CPU**, dominated by the MC Dropout model's 50 stochastic forward passes per test batch.

`test.py` produces a printed comparison table and saves the following figures:

File	Description
figures/best_baseline_predictions.png	One-week test slice — Deterministic LSTM predictions vs. ground truth (MW).
figures/best_baseline_training_curve.png	Deterministic training/validation loss vs. epoch.
figures/gaussian_predictions_with_uncertainty.png	Gaussian LSTM predictions with 68 % / 95 % bands.
figures/gaussian_calibration.png	Gaussian calibration plot after temperature scaling.
figures/mc_dropout_predictions_with_uncertainty.png	MC Dropout predictions with 68 % / 95 % bands.
figures/mc_dropout_calibration.png	MC Dropout calibration plot after temperature scaling.

The printed final table reports RMSE, NLL, CRPS and calibration error in original MW units.

Headline test-set metrics (2023–2025, 2015–2017 train, +temperature)

These numbers were reproduced from the shipped weights on a CPU-only machine using exactly the three packages above (run on 2026-04-10):

Model	Test RMSE (MW)	NLL (calibrated)	Test CRPS (MW)	Calibration error (after T)
Deterministic LSTM	5 014	-	-	-
Gaussian LSTM (T = 1.80)	5 267	0.346	3 148	0.231
MC Dropout LSTM (T = 9.15)	5 321	0.694	3 325	0.144

The Deterministic value is the **C8 sweep winner** (hidden=64 , 2 layers, dropout 0.2, weight decay 1e-4, scheduler on, train years 2015–2017, +temperature) and is an exact match against saved_models/sweep_results.json . All three models share that data path so the de-normalisation scaler in saved_models/scaler_params_best.json is consistent across them.

The Gaussian and MC Dropout models are trained for point accuracy (RMSE) and then **post-hoc calibrated** by fitting a single temperature scalar T on the validation set (src/evaluate.find_temperature). The raw Gaussian σ is overconfident (raw cal error ≈ 0.39), so $T \approx 1.80$ widens the intervals; the MC Dropout sample variance dramatically underestimates uncertainty (raw cal error ≈ 0.49), so $T \approx 9.15$ multiplies the spread to recover useful intervals.

3. Full reproduction — train all three models from scratch

To regenerate every model weight file from the raw demand CSVs:

```
micromamba activate comp0197-pt
cd <submission folder>
python train.py
```

train.py is a thin dispatcher; it sequentially runs:

- 1. train_deterministic.py — DeterministicLSTM, MSE loss, saves saved_models/deterministic_lstm.pt (+ training_history.json, scaler_params.json).
- 2. train_gaussian.py — GaussianLSTM, NLL loss with softplus σ , saves saved_models/gaussian_lstm.pt (+ gaussian_training_history.json, gaussian_temperature_T.json).
- 3. train_mc_dropout.py — MCDropoutLSTM, MSE loss with dropout staying on at test time, saves saved_models/mc_dropout_lstm.pt (+ mc_dropout_training_history.json, mc_dropout_temperature_T.json).

All three trainers default to the C8 sweep winner config: 2015–2017 train, 2018–2022 val, 2023–2025 test, sequence length 336 (one week of half-hourly data), hidden size 64, 2 LSTM layers, batch size 64, Adam (lr=1e-3, wd=1e-4), early stopping on validation loss with patience 10, max 100 epochs.

After train.py finishes, run python test.py again to evaluate the freshly-trained weights with the same script as in section 2.

Expected total runtime on CPU: **roughly 1–2 hours** depending on the machine, dominated by the MC Dropout training run.

4. Reproducing the ablation studies

Each ablation script is fully self-contained: it does its own data loading, training, evaluation, JSON output and figure rendering. Results JSONs and figures from the original runs are already included in saved_models/ and figures/ respectively, so a marker only needs to re-run an ablation if they want to verify a specific

number.

Script	Question answered	Outputs
experiments/baseline_sweep.py	Which deterministic config minimises validation RMSE? (Group A regularisation, Group B training years, Group C architecture.)	saved_models/sweep_results.json , figures/baseline_sweep_comparison.png , figures/best_baseline_*.png , saved_models/scaler_params_best.json , saved_models/deterministic_lstm_best.pt .
experiments/ablation_temperature.py	Does adding the London 2 m temperature feature improve all three models?	saved_models/ablation_temperature_results.json , figures/ablation_temperature.png .
experiments/ablation_seq_length.py	What is the best input window length (2 days / 3.5 days / 1 week / 2 weeks)?	saved_models/ablation_seq_length_results.json , figures/ablation_seq_length.png .
experiments/ablation_dropout_rate.py	Best dropout rate for the MC Dropout model (0.1 / 0.2 / 0.3 / 0.5)?	saved_models/ablation_dropout_rate_results.json , figures/ablation_dropout_rate.png .
experiments/ablation_holiday.py	Does an explicit is_holiday binary flag improve forecasts?	saved_models/ablation_holiday_results.json , figures/ablation_holiday.png .
experiments/ablation_lag_features.py	How much do explicit lagged demand features (yesterday / last week / month / year) improve forecasts?	saved_models/ablation_lag_features_results.json , figures/ablation_lag_features.png .
experiments/ablation_extended_analysis.py	Combined visual summary of the holiday and lag-feature ablations.	figures/ken_*.png .

Run any of them with, e.g.

```
python experiments/ablation_lag_features.py
```

Key reported ablation findings

- **Temperature feature** improves the Deterministic baseline test RMSE from ~5 635 MW (no temp) to ~5 263 MW (+temp) and similarly tightens Gaussian / MC Dropout intervals (saved_models/ablation_temperature_results.json).
- **Sequence length** of one week (336 half-hours) sits at the elbow; 2 weeks marginally improves val RMSE (~3 494) but at ~30 000 s vs. ~6 700 s training time (saved_models/ablation_seq_length_results.json).
- **MC Dropout rate** sweet spot is ~0.3 (test RMSE 5 166 MW, calibrated error 0.121); rate 0.5 over-regularises (saved_models/ablation_dropout_rate_results.json).
- **Lag features** (saved_models/ablation_lag_features_results.json) are the most impactful single change in the entire study:
 - No lags: Deterministic test RMSE ≈ 5 399 MW.
 - Add yesterday (lag_48): Deterministic RMSE drops to ≈ 2 625 MW (~51 % reduction).
 - Add last week (lag_336) on top: ≈ 2 504 MW (best).
 - Adding month (lag_1440) and year (lag_17520) makes things slightly worse (≈ 2 796 MW) — diminishing/negative returns from very long lags. MC Dropout calibration error drops from 0.104 to 0.029 with the full lag set.
- **Holiday flag** gives a small but consistent RMSE improvement for Deterministic and Gaussian (~5 305 → ~5 201 MW; ~5 466 → ~5 252 MW) without measurably changing calibration (saved_models/ablation_holiday_results.json).

5. File index

.	
├─ README.md	High-level project overview
├─ instruction.pdf	This document (rendered)
├─ instruction.md	Source markdown for instruction.pdf
├─ train.py	Dispatcher: trains all three models
├─ train_deterministic.py	Deterministic LSTM training script
├─ train_gaussian.py	Gaussian LSTM training script
├─ train_mc_dropout.py	MC Dropout LSTM training script
├─ test.py	Loads saved weights and reports all metrics + figures
├─ recalibrate_gaussian.py	Re-fits the Gaussian temperature scaler T
├─ configs/default.yaml	Hyperparameter defaults
├─ src/	
│ └─ data_loader.py	Demand CSV loading + feature engineering + sliding-window Dataset
│ └─ preprocessing.py	(Reserved – preprocessing is integrated into data_loader.py)
│ └─ EDA.py	Exploratory data analysis figure script
│ └─ evaluate.py	RMSE / NLL / CRPS / calibration metrics + plotting helpers
│ └─ models/	
│ │ └─ base_lstm.py	Shared LSTM encoder
│ │ └─ deterministic.py	DeterministicLSTM (point prediction, MSE)
│ │ └─ gaussian.py	GaussianLSTM (μ , σ via softplus, NLL loss)
│ │ └─ mc_dropout.py	MCDropoutLSTM (dropout-on at test time)
├─ experiments/	All ablation scripts (see section 4)
├─ data/	
│ └─ processed/data/demanddata_<year>.csv	UK ESO half-hourly demand 2001–2025
│ └─ raw/temperature_london.csv	Cached London 2 m temperature 2001–2025 (Open-Meteo)
├─ saved_models/	
│ └─ deterministic_lstm.pt	Final trained Deterministic weights (hidden=64, 2 layers)
│ └─ gaussian_lstm.pt	Final trained Gaussian weights
│ └─ mc_dropout_lstm.pt	Final trained MC Dropout weights
│ └─ scaler_params_best.json	Z-score scaler used at inference time (loaded by test.py)
│ └─ scaler_params.json	Scaler from train_deterministic.py (regenerated on each run)
│ └─ training_history.json	Deterministic per-epoch losses
│ └─ gaussian_training_history.json	Gaussian per-epoch losses
│ └─ mc_dropout_training_history.json	MC Dropout per-epoch losses
│ └─ gaussian_temperature_T.json	Gaussian post-hoc calibration temperature T
│ └─ mc_dropout_temperature_T.json	MC Dropout post-hoc calibration temperature T
│ └─ sweep_results.json	Output of experiments/baseline_sweep.py
│ └─ ablation_temperature_results.json	
│ └─ ablation_seq_length_results.json	
│ └─ ablation_dropout_rate_results.json	
│ └─ ablation_holiday_results.json	
│ └─ ablation_lag_features_results.json	
├─ figures/	All figures referenced in the report
└─ docs/plans/	Design plans for the holiday and lag ablations

6. GenAI declaration

The team used Claude (Anthropic) as a coding assistant during development. Claude was used to:

- Sketch boilerplate for some training and evaluation scripts (notably `test.py` and the closed-form CRPS in `src/evaluate.py`).
- Help structure and review the hyperparameter sweep and the holiday / lag-features ablation experiments.

All technical decisions, model architectures, hyperparameter choices, and reported results were verified and run by the team. Each model implementation in `src/models/` was authored, reviewed and debugged by a named team member. All metrics in this document and the report were produced by the included scripts on the team's hardware, not by any AI tool.